1. Aparece uma caixinha para digitar o nome. Digite o caminho completo exatamente como está neste guia, por exemplo:
   `backend/pom.xml`
2. Aperte Enter. O arquivo abre vazio.
3. Cole o código (Ctrl+V) e salve (Ctrl+S).

Truque: quando você digita uma barra `/` no nome, o VS Code cria as pastas sozinho. Digitar `backend/src/main/resources/application.properties` já cria `backend`, `src`, `main` e `resources`. Você não precisa criar pastas à mão.

A.5 Três coisas que confundem

1. O nome tem que ser idêntico, com maiúsculas e minúsculas. `Lote.java` é diferente de `lote.java`.
2. Pastas "coladas": o VS Code mostra `com / senai / rastreabilidade` numa linha só (ex.: `com.senai.rastreabilidade`). Isso é normal: são 3 pastas juntas para economizar espaço.
3. Erros vermelhos: enquanto o Maven não baixar as bibliotecas (Parte 4 do guia anterior), os arquivos Java mostram sublinhado vermelho. Isso some sozinho depois.

A.6 Como ficará o Explorer no final

```
rastreabilidade-calcados
├── backend
│   ├── pom.xml
│   └── src/main
│       ├── resources/application.properties
│       └── java/com/senai/rastreabilidade
│           ├── RastreabilidadeApplication.java
│           ├── config/   → 3 arquivos
│           ├── dto/      → 1 arquivo
│           ├── model/    → 8 arquivos
│           ├── repository/ → 5 arquivos
│           ├── service/  → 2 arquivos
│           └── web/      → 2 arquivos
└── frontend
    ├── index.html, package.json, vite.config.js
    └── src
        ├── main.js, api.js, style.css, App.vue
        ├── router/index.js
        └── views/ → 4 arquivos
```

PARTE B: Os arquivos do BACKEND (23 arquivos)

Para cada um: crie o arquivo com o nome indicado e cole o código logo abaixo.

Arquivo 1: `backend/pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
  </parent>
  <groupId>com.senai</groupId>
  <artifactId>rastreabilidade</artifactId>
  <version>1.0.0</version>
  <properties><java.version>17</java.version></properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.security</groupId>
      <artifactId>spring-security-crypto</artifactId>
    </dependency>
    <dependency>
      <groupId>org.mariadb.jdbc</groupId>
      <artifactId>mariadb-java-client</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
```

```
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>app</finalName>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <excludes>
            <exclude>
              <groupId>org.projectlombok</groupId>
              <artifactId>lombok</artifactId>
            </exclude>
          </excludes>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Arquivo 2: `backend/src/main/resources/application.properties`

```
server.port=8080

# Banco do XAMPP (usuario root, SEM senha, porta 3306)
spring.datasource.url=jdbc:mariadb://localhost:3306/rastreabilidade
spring.datasource.username=root
spring.datasource.password=

# O banco foi criado pelo script SQL, entao o Hibernate nao mexe nas tabelas
spring.jpa.hibernate.ddl-auto=none
spring.jpa.open-in-view=false

# Endereco do site liberado para acessar a API
app.cors.origins=http://localhost:5173
```

Arquivo 3: `backend/src/main/java/com/senai/rastreabilidade/RastreabilidadeApplication.java`

```
package com.senai.rastreabilidade;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RastreabilidadeApplication {
   public static void main(String[] args) {
      SpringApplication.run(RastreabilidadeApplication.class, args);
   }
}
```

Arquivo 4:
backend/src/main/java/com/senai/rastreabilidade/model/StatusLote.java

```
package com.senai.rastreabilidade.model;

public enum StatusLote {
   CRIADO, ENVIADO_BANCA, EM_PROCESSAMENTO, CONCLUIDO;

   /** Cada leitura do QR Code avanca UM passo no fluxo. */
   public StatusLote proximo() {
      return switch (this) {
         case CRIADO -> ENVIADO_BANCA;
         case ENVIADO_BANCA -> EM_PROCESSAMENTO;
         case EM_PROCESSAMENTO -> CONCLUIDO;
         case CONCLUIDO -> throw new IllegalStateException("Lote ja concluido");
      };
   }
}
```

Arquivo 5:
backend/src/main/java/com/senai/rastreabilidade/model/TipoPerda.java

```
package com.senai.rastreabilidade.model;

public enum TipoPerda { DANIFICADA, EXTRAVIADA, DEFEITO_FABRICACAO }
```

Arquivo 6:
backend/src/main/java/com/senai/rastreabilidade/model/Perfil.java

```
package com.senai.rastreabilidade.model;
```

```
public enum Perfil { GESTOR, OPERADOR }
```

Arquivo 7:
backend/src/main/java/com/senai/rastreabilidade/model/Usuario.java

```
package com.senai.rastreabilidade.model;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Entity @Table(name = "usuarios") @Getter @Setter
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String email;
    private String senhaHash;
    @Enumerated(EnumType.STRING)
    private Perfil perfil;
    private boolean ativo;
}
```

Arquivo 8:
backend/src/main/java/com/senai/rastreabilidade/model/Banca.java

```
package com.senai.rastreabilidade.model;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Entity @Table(name = "bancas") @Getter @Setter
public class Banca {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String responsavel;
    private String telefone;
    private String cidade;
    private boolean ativa;
}
```

Arquivo 9:

backend/src/main/java/com/senai/rastreabilidade/model/Lote.java

```
package com.senai.rastreabilidade.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import lombok.Getter;
import lombok.Setter;

@Entity @Table(name = "lotes") @Getter @Setter
public class Lote {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true, length = 36)
    private String codigo;
    private String modelo;
    private int quantidade;
    private BigDecimal valorUnitario;
    @ManyToOne(optional = false) @JoinColumn(name = "banca_id")
    private Banca banca;
    @ManyToOne(optional = false) @JoinColumn(name = "criado_por")
    private Usuario criadoPor;
    @Enumerated(EnumType.STRING)
    private StatusLote status;
    private LocalDateTime criadoEm;
    private LocalDateTime enviadoEm;
    private LocalDateTime recebidoEm;
    private LocalDateTime concluidoEm;
}
```

Arquivo 10:

backend/src/main/java/com/senai/rastreabilidade/model/Movimentacao.java

```
package com.senai.rastreabilidade.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import lombok.Getter;
import lombok.Setter;
```

```
@Entity @Table(name = "movimentacoes") @Getter @Setter
public class Movimentacao {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne(optional = false) @JoinColumn(name = "lote_id")
    private Lote lote;
    @ManyToOne(optional = false) @JoinColumn(name = "usuario_id")
    private Usuario usuario;
    @Enumerated(EnumType.STRING)
    private StatusLote statusAnterior;
    @Enumerated(EnumType.STRING)
    private StatusLote statusNovo;
    private LocalDateTime dataHora;
    private String observacao;
}
```

Arquivo 11:
backend/src/main/java/com/senai/rastreabilidade/model/Perda.java

```
package com.senai.rastreabilidade.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import lombok.Getter;
import lombok.Setter;

@Entity @Table(name = "perdas") @Getter @Setter
public class Perda {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne(optional = false) @JoinColumn(name = "lote_id")
    private Lote lote;
    @ManyToOne(optional = false) @JoinColumn(name = "usuario_id")
    private Usuario usuario;
    @Enumerated(EnumType.STRING)
    private TipoPerda tipo;
    private int quantidade;
    private String observacao;
    private LocalDateTime registradaEm;
}
```

Arquivo 12:
backend/src/main/java/com/senai/rastreabilidade/repository/UsuarioRepository.java

```
package com.senai.rastreabilidade.repository;

import com.senai.rastreabilidade.model.Usuario;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByEmail(String email);
}
```

Arquivo 13:
backend/src/main/java/com/senai/rastreabilidade/repository/BancaRepository.java

```
package com.senai.rastreabilidade.repository;

import com.senai.rastreabilidade.model.Banca;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BancaRepository extends JpaRepository<Banca, Long> { }
```

Arquivo 14:
backend/src/main/java/com/senai/rastreabilidade/repository/LoteRepository.java

```
package com.senai.rastreabilidade.repository;

import com.senai.rastreabilidade.model.Lote;
import com.senai.rastreabilidade.model.StatusLote;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface LoteRepository extends JpaRepository<Lote, Long> {
    Optional<Lote> findByCodigo(String codigo);
    long countByStatus(StatusLote status);
}
```

Arquivo 15:
backend/src/main/java/com/senai/rastreabilidade/repository/MovimentacaoRepository.java

```
package com.senai.rastreabilidade.repository;

import com.senai.rastreabilidade.model.Movimentacao;
```

```
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MovimentacaoRepository extends JpaRepository<Movimentacao,
Long> {
   List<Movimentacao> findByLoteIdOrderByDataHoraDesc(Long loteId);
   List<Movimentacao> findTop10ByOrderByDataHoraDesc();
}
```

Arquivo 16:

backend/src/main/java/com/senai/rastreabilidade/repository/PerdaRepository.java

```
package com.senai.rastreabilidade.repository;

import com.senai.rastreabilidade.model.Perda;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface PerdaRepository extends JpaRepository<Perda, Long> {
   @Query("select coalesce(sum(p.quantidade), 0) from Perda p where p.lote.id = :id")
   long somaPorLote(@Param("id") Long id);

   @Query("select coalesce(sum(p.quantidade), 0) from Perda p")
   long somaTotal();
}
```

Arquivo 17:

backend/src/main/java/com/senai/rastreabilidade/dto/Dtos.java

```
package com.senai.rastreabilidade.dto;

import com.senai.rastreabilidade.model.*;
import jakarta.validation.constraints.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

public class Dtos {
   public record LoginRequest(@NotBlank String email, @NotBlank String senha) { }
   public record LoginResponse(String token, String nome, String perfil) { }
```

```
    public record LoteRequest(@NotBlank String modelo,
                              @Min(1) int quantidade,
                              @NotNull @DecimalMin("0.0") BigDecimal valorUnitario,
                              @NotNull Long bancaId) { }

    public record PerdaRequest(@NotNull TipoPerda tipo,
                               @Min(1) int quantidade,
                               String observacao) { }

    public record LoteResponse(Long id, String codigo, String modelo, int quantidade,
                               BigDecimal valorUnitario, String banca, StatusLote status,
                               int totalPerdas, BigDecimal valorDevido, Long cicloMinutos,
                               LocalDateTime criadoEm) { }

    public record MovResponse(String lote, StatusLote statusAnterior, StatusLote
statusNovo,
                              String usuario, LocalDateTime dataHora, String observacao) { }

    public record DashboardResponse(long total, Map<String, Long> porStatus,
                                    long totalPerdas, BigDecimal valorDevido,
                                    double cicloMedioMinutos, List<MovResponse> ultimas) { }
}
```

Arquivo 18:

backend/src/main/java/com/senai/rastreabilidade/service/AuthService.java

```
package com.senai.rastreabilidade.service;

import com.senai.rastreabilidade.dto.Dtos.*;
import com.senai.rastreabilidade.model.Usuario;
import com.senai.rastreabilidade.repository.UsuarioRepository;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

@Service
public class AuthService {
    private final UsuarioRepository repo;
    private final BCryptPasswordEncoder enc = new BCryptPasswordEncoder();
    private final Map<String, Long> tokens = new ConcurrentHashMap<>();

    public AuthService(UsuarioRepository repo) { this.repo = repo; }
```

```
    public LoginResponse login(LoginRequest r) {
        Usuario u = repo.findByEmail(r.email())
            .filter(Usuario::isAtivo)
            .filter(x -> enc.matches(r.senha(), x.getSenhaHash()))
            .orElseThrow(() -> new ResponseStatusException(
                HttpStatus.UNAUTHORIZED, "E-mail ou senha invalidos"));
        String token = UUID.randomUUID().toString();
        tokens.put(token, u.getId());
        return new LoginResponse(token, u.getNome(), u.getPerfil().name());
    }

    public Optional<Usuario> porToken(String token) {
        return Optional.ofNullable(tokens.get(token)).flatMap(repo::findById);
    }

    public String hash(String senha) { return enc.encode(senha); }
}
```

Arquivo 19:
backend/src/main/java/com/senai/rastreabilidade/service/LoteService.java

```
package com.senai.rastreabilidade.service;

import com.senai.rastreabilidade.dto.Dtos.*;
import com.senai.rastreabilidade.model.*;
import com.senai.rastreabilidade.repository.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.*;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class LoteService {
    private final LoteRepository lotes;
    private final BancaRepository bancas;
    private final MovimentacaoRepository movs;
    private final PerdaRepository perdas;

    public LoteService(LoteRepository lotes, BancaRepository bancas,
                       MovimentacaoRepository movs, PerdaRepository perdas) {
        this.lotes = lotes; this.bancas = bancas; this.movs = movs; this.perdas = perdas;
    }
```

```
    @Transactional
    public LoteResponse criar(LoteRequest r, Usuario u) {
        Banca b = bancas.findById(r.bancaId())
            .orElseThrow(() -> new NoSuchElementException("Banca nao encontrada"));
        Lote l = new Lote();
        l.setCodigo(UUID.randomUUID().toString());
        l.setModelo(r.modelo());
        l.setQuantidade(r.quantidade());
        l.setValorUnitario(r.valorUnitario());
        l.setBanca(b);
        l.setCriadoPor(u);
        l.setStatus(StatusLote.CRIADO);
        l.setCriadoEm(LocalDateTime.now());
        lotes.save(l);
        registrar(l, null, StatusLote.CRIADO, u, "Lote criado");
        return toResponse(l);
    }

    @Transactional
    public LoteResponse lerQr(String codigo, Usuario u) {
        Lote l = buscar(codigo);
        StatusLote antigo = l.getStatus();
        StatusLote novo = antigo.proximo();
        LocalDateTime agora = LocalDateTime.now();
        switch (novo) {
            case ENVIADO_BANCA -> l.setEnviadoEm(agora);
            case EM_PROCESSAMENTO -> l.setRecebidoEm(agora);
            case CONCLUIDO -> l.setConcluidoEm(agora);
            default -> { }
        }
        l.setStatus(novo);
        lotes.save(l);
        registrar(l, antigo, novo, u, "Leitura de QR Code");
        return toResponse(l);
    }

    @Transactional
    public void registrarPerda(String codigo, PerdaRequest r, Usuario u) {
        Lote l = buscar(codigo);
        if (l.getStatus() != StatusLote.EM_PROCESSAMENTO)
            throw new IllegalStateException(
                "Perdas so podem ser registradas com o lote Em Processamento");
        if (perdas.somaPorLote(l.getId()) + r.quantidade() > l.getQuantidade())
            throw new IllegalArgumentException("Perdas maiores que a quantidade do
lote");
        Perda p = new Perda();
        p.setLote(l); p.setUsuario(u); p.setTipo(r.tipo());
```

```
        p.setQuantidade(r.quantidade()); p.setObservacao(r.observacao());
        p.setRegistradaEm(LocalDateTime.now());
        perdas.save(p);
    }

    @Transactional(readOnly = true)
    public List<LoteResponse> listar() {
        return lotes.findAll(Sort.by(Sort.Direction.DESC, "criadoEm"))
            .stream().map(this::toResponse).toList();
    }

    @Transactional(readOnly = true)
    public LoteResponse obter(String codigo) { return toResponse(buscar(codigo)); }

    @Transactional(readOnly = true)
    public List<MovResponse> historico(String codigo) {
        Lote l = buscar(codigo);
        return movs.findByLoteIdOrderByDataHoraDesc(l.getId())
            .stream().map(this::toMov).toList();
    }

    @Transactional(readOnly = true)
    public DashboardResponse dashboard() {
        Map<String, Long> porStatus = new LinkedHashMap<>();
        for (StatusLote s : StatusLote.values())
            porStatus.put(s.name(), lotes.countByStatus(s));
        BigDecimal devido = BigDecimal.ZERO;
        long somaCiclo = 0, concluidos = 0;
        List<Lote> todos = lotes.findAll();
        for (Lote l : todos) {
            LoteResponse r = toResponse(l);
            devido = devido.add(r.valorDevido());
            if (r.cicloMinutos() != null) { somaCiclo += r.cicloMinutos(); concluidos++; }
        }
        double media = concluidos == 0 ? 0 : (double) somaCiclo / concluidos;
        var ultimas =
movs.findTop10ByOrderByDataHoraDesc().stream().map(this::toMov).toList();
        return new DashboardResponse(todos.size(), porStatus, perdas.somaTotal(),
            devido, media, ultimas);
    }

    private Lote buscar(String codigo) {
        return lotes.findByCodigo(codigo)
            .orElseThrow(() -> new NoSuchElementException("Lote nao encontrado"));
    }

    private void registrar(Lote l, StatusLote ant, StatusLote novo, Usuario u, String obs)
{
```

```
        Movimentacao m = new Movimentacao();
        m.setLote(l); m.setUsuario(u);
        m.setStatusAnterior(ant); m.setStatusNovo(novo);
        m.setDataHora(LocalDateTime.now()); m.setObservacao(obs);
        movs.save(m);
    }

    private LoteResponse toResponse(Lote l) {
        int perdidas = (int) perdas.somaPorLote(l.getId());
        BigDecimal devido = BigDecimal.ZERO;
        Long ciclo = null;
        if (l.getStatus() == StatusLote.CONCLUIDO) {
            devido = l.getValorUnitario()
                .multiply(BigDecimal.valueOf(l.getQuantidade() - perdidas));
            ciclo = Duration.between(l.getEnviadoEm(), l.getConcluidoEm()).toMinutes();
        }
        return new LoteResponse(l.getId(), l.getCodigo(), l.getModelo(), l.getQuantidade(),
            l.getValorUnitario(), l.getBanca().getNome(), l.getStatus(), perdidas,
            devido, ciclo, l.getCriadoEm());
    }

    private MovResponse toMov(Movimentacao m) {
        return new MovResponse(m.getLote().getModelo(), m.getStatusAnterior(),
            m.getStatusNovo(), m.getUsuario().getNome(), m.getDataHora(),
            m.getObservacao());
    }
}
```

Arquivo 20:

backend/src/main/java/com/senai/rastreabilidade/config/AuthInterceptor.java

```
package com.senai.rastreabilidade.config;

import com.senai.rastreabilidade.service.AuthService;
import jakarta.servlet.http.*;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class AuthInterceptor implements HandlerInterceptor {
    private final AuthService auth;

    public AuthInterceptor(AuthService auth) { this.auth = auth; }

    @Override
```

```
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object
h)
            throws Exception {
        if (req.getMethod().equals("OPTIONS")) return true;
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            var usuario = auth.porToken(header.substring(7));
            if (usuario.isPresent()) {
                req.setAttribute("usuario", usuario.get());
                return true;
            }
        }
        res.setStatus(401);
        res.setContentType("application/json");
        res.getWriter().write("{\"erro\":\"Nao autenticado\"}");
        return false;
    }
}
```

Arquivo 21:

backend/src/main/java/com/senai/rastreabilidade/config/WebConfig.java

```
package com.senai.rastreabilidade.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.*;

@Configuration
public class WebConfig implements WebMvcConfigurer {
    private final AuthInterceptor interceptor;
    @Value("${app.cors.origins}") private String origins;

    public WebConfig(AuthInterceptor interceptor) { this.interceptor = interceptor; }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(interceptor)
            .addPathPatterns("/api/**")
            .excludePathPatterns("/api/auth/login");
    }

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
```

```
            .allowedOrigins(origins.split(","))
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*");
    }
}
```

Arquivo 22:
backend/src/main/java/com/senai/rastreabilidade/config/DataSeeder.java

```
package com.senai.rastreabilidade.config;

import com.senai.rastreabilidade.model.*;
import com.senai.rastreabilidade.repository.*;
import com.senai.rastreabilidade.service.AuthService;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

/** Cria usuarios e bancas de exemplo na primeira execucao (senha: 123456). */
@Component
public class DataSeeder implements CommandLineRunner {
    private final UsuarioRepository usuarios;
    private final BancaRepository bancas;
    private final AuthService auth;

    public DataSeeder(UsuarioRepository usuarios, BancaRepository bancas,
AuthService auth) {
        this.usuarios = usuarios; this.bancas = bancas; this.auth = auth;
    }

    @Override
    public void run(String... args) {
        if (usuarios.count() == 0) {
            usuarios.save(novo("Gestor SENAI", "gestor@senai.com", Perfil.GESTOR));
            usuarios.save(novo("Operador SENAI", "operador@senai.com",
Perfil.OPERADOR));
        }
        if (bancas.count() == 0) {
            bancas.save(banca("Banca Silva", "Sr. Silva"));
            bancas.save(banca("Banca Dona Rita", "Rita Souza"));
            bancas.save(banca("Banca Tres Irmaos", "Carlos Pereira"));
        }
    }

    private Usuario novo(String nome, String email, Perfil perfil) {
        Usuario u = new Usuario();
```

```
        u.setNome(nome); u.setEmail(email); u.setPerfil(perfil);
        u.setSenhaHash(auth.hash("123456")); u.setAtivo(true);
        return u;
    }

    private Banca banca(String nome, String responsavel) {
        Banca b = new Banca();
        b.setNome(nome); b.setResponsavel(responsavel);
        b.setCidade("Nova Serrana"); b.setAtiva(true);
        return b;
    }
}
```

Arquivo 23:

backend/src/main/java/com/senai/rastreabilidade/web/ApiController.java

```
package com.senai.rastreabilidade.web;

import com.senai.rastreabilidade.dto.Dtos.*;
import com.senai.rastreabilidade.model.*;
import com.senai.rastreabilidade.repository.BancaRepository;
import com.senai.rastreabilidade.service.*;
import jakarta.validation.Valid;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/api")
public class ApiController {
    private final AuthService auth;
    private final LoteService service;
    private final BancaRepository bancas;

    public ApiController(AuthService auth, LoteService service, BancaRepository
bancas) {
        this.auth = auth; this.service = service; this.bancas = bancas;
    }

    @PostMapping("/auth/login")
    public LoginResponse login(@Valid @RequestBody LoginRequest r) { return
auth.login(r); }

    @GetMapping("/bancas")
```

```
  public List<Banca> listarBancas() { return bancas.findAll(); }

  @PostMapping("/bancas")
  public Banca novaBanca(@RequestBody Banca b, @RequestAttribute("usuario")
Usuario u) {
    exigeGestor(u);
    b.setId(null); b.setAtiva(true);
    return bancas.save(b);
  }

  @GetMapping("/lotes")
  public List<LoteResponse> listarLotes() { return service.listar(); }

  @PostMapping("/lotes")
  public LoteResponse criarLote(@Valid @RequestBody LoteRequest r,
                    @RequestAttribute("usuario") Usuario u) {
    exigeGestor(u);
    return service.criar(r, u);
  }

  @GetMapping("/lotes/{codigo}")
  public LoteResponse obter(@PathVariable String codigo) { return
service.obter(codigo); }

  @PostMapping("/lotes/{codigo}/scan")
  public LoteResponse scan(@PathVariable String codigo,
                  @RequestAttribute("usuario") Usuario u) {
    return service.lerQr(codigo, u);
  }

  @PostMapping("/lotes/{codigo}/perdas") @ResponseStatus(HttpStatus.CREATED)
  public void perda(@PathVariable String codigo, @Valid @RequestBody
PerdaRequest r,
            @RequestAttribute("usuario") Usuario u) {
    service.registrarPerda(codigo, r, u);
  }

  @GetMapping("/lotes/{codigo}/movimentacoes")
  public List<MovResponse> historico(@PathVariable String codigo) {
    return service.historico(codigo);
  }

  @GetMapping("/dashboard")
  public DashboardResponse dashboard() { return service.dashboard(); }

  private void exigeGestor(Usuario u) {
    if (u.getPerfil() != Perfil.GESTOR)
```

```
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Apenas o
gestor pode isso");
    }
}
```

Arquivo 24:

backend/src/main/java/com/senai/rastreabilidade/web/ErroHandler.java

```
package com.senai.rastreabilidade.web;

import java.util.*;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

@RestControllerAdvice
public class ErroHandler {
    private ResponseEntity<Map<String, String>> resp(HttpStatus s, String msg) {
        return ResponseEntity.status(s).body(Map.of("erro", msg == null ? "Erro" : msg));
    }

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<Map<String, String>> status(ResponseStatusException e) {
        return resp(HttpStatus.valueOf(e.getStatusCode().value()), e.getReason());
    }

    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<Map<String, String>>
naoEncontrado(NoSuchElementException e) {
        return resp(HttpStatus.NOT_FOUND, e.getMessage());
    }

    @ExceptionHandler({IllegalStateException.class, IllegalArgumentException.class})
    public ResponseEntity<Map<String, String>> regra(RuntimeException e) {
        return resp(HttpStatus.BAD_REQUEST, e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
validacao(MethodArgumentNotValidException e) {
        return resp(HttpStatus.BAD_REQUEST, "Dados invalidos: confira os campos");
    }
}
```

Contei 24 arquivos no backend (o 31 que citei no começo era 24 + 12 do frontend = 36 no total; foi erro de conta meu). Confira a lista pela pasta no Explorer.

PARTE C: Os arquivos do FRONTEND (12 arquivos)

Arquivo 25: `frontend/package.json`

```
{
  "name": "frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.7.7",
    "html5-qrcode": "^2.3.8",
    "qrcode": "^1.5.4",
    "vue": "^3.5.12",
    "vue-router": "^4.4.5"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^5.1.4",
    "vite": "^5.4.10"
  }
}
```

Arquivo 26: `frontend/vite.config.js`

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: { port: 5173 },
})
```

Arquivo 27: `frontend/index.html`

```
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Rastreabilidade de Calçados</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

Arquivo 28: `frontend/src/main.js`

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import './style.css'

createApp(App).use(router).mount('#app')
```

Arquivo 29: `frontend/src/api.js`

```
import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8080/api',
})

api.interceptors.request.use(cfg => {
  const t = localStorage.getItem('token')
  if (t) cfg.headers.Authorization = 'Bearer ' + t
  return cfg
})

api.interceptors.response.use(r => r, e => {
  if (e.response && e.response.status === 401 && !e.config.url.includes('/auth/login')) {
    localStorage.clear()
    location.href = '/login'
  }
  return Promise.reject(e)
})

export const msgErro = e => e.response?.data?.erro || 'Erro ao falar com o servidor'
export default api
```

Arquivo 30: `frontend/src/style.css`

```
:root { --pr:#c8102e; --bg:#f5f6f8; --card:#fff; --bd:#e3e6ea; --tx:#1b1f27; }
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: var(--bg); color:
var(--tx); }
header { background: var(--pr); color: #fff; padding: 10px 16px; display: flex;
  justify-content: space-between; flex-wrap: wrap; gap: 8px; }
nav a { color: #fff; margin-left: 14px; text-decoration: none; }
nav a.router-link-exact-active { text-decoration: underline; font-weight: 700; }
main { max-width: 1000px; margin: 0 auto; padding: 16px; }
.card { background: var(--card); border: 1px solid var(--bd); border-radius: 12px;
  padding: 16px; margin-bottom: 16px; }
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 12px; margin-bottom: 16px; }
.kpi { background: var(--card); border: 1px solid var(--bd); border-radius: 12px;
padding: 12px; }
.kpi b { display: block; font-size: 24px; }
.row { display: flex; gap: 16px; flex-wrap: wrap; }
.row > * { flex: 1; min-width: 260px; }
input, select { width: 100%; padding: 9px; margin: 4px 0 10px; border: 1px solid
var(--bd);
  border-radius: 8px; }
button { background: var(--pr); color: #fff; border: 0; border-radius: 8px;
  padding: 9px 16px; cursor: pointer; }
.tw { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; min-width: 520px; }
th, td { text-align: left; padding: 8px; border-bottom: 1px solid var(--bd); }
.login { max-width: 340px; margin: 10vh auto; }
.erro { color: #b91c1c; }
#reader { width: 100%; max-width: 420px; }
@media print { header, button, .row > .card:first-child { display: none; } }
```

Arquivo 31: `frontend/src/App.vue`

```
<script setup>
import { useRoute, useRouter } from 'vue-router'
const route = useRoute()
const router = useRouter()

function sair() {
  localStorage.clear()
  router.push('/login')
}
</script>
```

```
<template>
  <header v-if="route.path !== '/login'">
    <b>Rastreabilidade de Calçados</b>
    <nav>
      <router-link to="/">Painel</router-link>
      <router-link to="/lotes">Lotes</router-link>
      <router-link to="/leitor">Leitor QR</router-link>
      <a href="#" @click.prevent="sair">Sair</a>
    </nav>
  </header>
  <main><router-view /></main>
</template>
```

Arquivo 32: `frontend/src/router/index.js`

```
import { createRouter, createWebHistory } from 'vue-router'
import Login from '../views/Login.vue'
import Dashboard from '../views/Dashboard.vue'
import Lotes from '../views/Lotes.vue'
import Leitor from '../views/Leitor.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/login', component: Login },
    { path: '/', component: Dashboard },
    { path: '/lotes', component: Lotes },
    { path: '/leitor', component: Leitor },
  ],
})

router.beforeEach(to => {
  if (to.path !== '/login' && !localStorage.getItem('token')) return '/login'
})

export default router
```

Arquivo 33: `frontend/src/views/Login.vue`

```
<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import api, { msgErro } from '../api'

const router = useRouter()
```

```
const email = ref('gestor@senai.com')
const senha = ref('123456')
const erro = ref('')

async function entrar() {
  try {
    const { data } = await api.post('/auth/login', { email: email.value, senha: senha.value
})
    localStorage.setItem('token', data.token)
    localStorage.setItem('nome', data.nome)
    localStorage.setItem('perfil', data.perfil)
    router.push('/')
  } catch (e) {
    erro.value = msgErro(e)
  }
}
</script>

<template>
  <div class="card login">
    <h2>Entrar</h2>
    <label>E-mail<input v-model="email"></label>
    <label>Senha<input v-model="senha" type="password"
@keyup.enter="entrar"></label>
    <button @click="entrar">Entrar</button>
    <p class="erro" v-if="erro">{{ erro }}</p>
  </div>
</template>
```

Arquivo 34: `frontend/src/views/Dashboard.vue`

```
<script setup>
import { ref, onMounted, onUnmounted } from 'vue'
import api from '../api'

const d = ref(null)
const lotes = ref([])
const nomes = {
  CRIADO: 'Criado',
  ENVIADO_BANCA: 'Enviado à Banca',
  EM_PROCESSAMENTO: 'Em Processamento',
  CONCLUIDO: 'Concluído/Na Fábrica',
}

async function carregar() {
  d.value = (await api.get('/dashboard')).data
  lotes.value = (await api.get('/lotes')).data
```

```
}

let timer
onMounted(() => { carregar(); timer = setInterval(carregar, 10000) })
onUnmounted(() => clearInterval(timer))
</script>

<template>
  <div v-if="d">
    <div class="grid">
      <div class="kpi"><span>Total de lotes</span><b>{{ d.total }}</b></div>
      <div class="kpi" v-for="(qtd, s) in d.porStatus" :key="s">
        <span>{{ nomes[s] }}</span><b>{{ qtd }}</b>
      </div>
      <div class="kpi"><span>Peças perdidas</span><b>{{ d.totalPerdas }}</b></div>
      <div class="kpi"><span>Valor devido</span><b>R$ {{
Number(d.valorDevido).toFixed(2) }}</b></div>
      <div class="kpi"><span>Ciclo médio (min)</span><b>{{
d.cicloMedioMinutos.toFixed(0) }}</b></div>
    </div>

    <div class="card tw">
      <h3>Lotes</h3>
      <table>

<tr><th>Modelo</th><th>Banca</th><th>Qtd</th><th>Status</th><th>Perdas</th><th
>Ciclo</th></tr>
        <tr v-for="l in lotes" :key="l.id">
          <td>{{ l.modelo }}</td><td>{{ l.banca }}</td><td>{{ l.quantidade }}</td>
          <td>{{ nomes[l.status] }}</td><td>{{ l.totalPerdas }}</td>
          <td>{{ l.cicloMinutos != null ? l.cicloMinutos + ' min' : '-' }}</td>
        </tr>
      </table>
    </div>

    <div class="card">
      <h3>Últimas movimentações</h3>
      <div v-for="(m, i) in d.ultimas" :key="i">
        {{ new Date(m.dataHora).toLocaleString('pt-BR') }} - <b>{{ m.lote }}</b>:
        {{ nomes[m.statusNovo] }} ({{ m.usuario }})
      </div>
    </div>
  </div>
</template>
```

Arquivo 35: `frontend/src/views/Lotes.vue`

```
<script setup>
import { ref, onMounted } from 'vue'
import QRCode from 'qrcode'
import api, { msgErro } from '../api'

const bancas = ref([])
const lotes = ref([])
const erro = ref('')
const qr = ref(null)
const f = ref({ modelo: '', quantidade: 100, valorUnitario: 1.5, bancaId: null })

async function carregar() {
  bancas.value = (await api.get('/bancas')).data
  lotes.value = (await api.get('/lotes')).data
  if (!f.value.bancaId && bancas.value.length) f.value.bancaId = bancas.value[0].id
}

async function mostrarQr(l) {
  qr.value = { lote: l, img: await QRCode.toDataURL(l.codigo, { width: 240 }) }
}

async function criar() {
  erro.value = ''
  try {
    const { data } = await api.post('/lotes', f.value)
    await mostrarQr(data)
    await carregar()
  } catch (e) {
    erro.value = msgErro(e)
  }
}

function imprimir() { window.print() }
onMounted(carregar)
</script>

<template>
  <div class="row">
    <div class="card">
      <h2>Novo lote</h2>
      <label>Modelo<input v-model="f.modelo"></label>
      <label>Quantidade (pares)<input v-model.number="f.quantidade" type="number"
min="1"></label>
      <label>Banca
        <select v-model="f.bancaId">
          <option v-for="b in bancas" :key="b.id" :value="b.id">{{ b.nome }}</option>
        </select>
      </label>
```

```
    <label>Valor por par (R$)
      <input v-model.number="f.valorUnitario" type="number" step="0.01">
    </label>
    <button @click="criar">Gerar lote e QR Code</button>
    <p class="erro" v-if="erro">{{ erro }}</p>
  </div>

  <div class="card" v-if="qr" style="text-align:center">
    <h2>{{ qr.lote.modelo }}</h2>
    <img :src="qr.img" alt="QR Code do lote">
    <p>{{ qr.lote.codigo }}</p>
    <button @click="imprimir">Imprimir etiqueta</button>
  </div>
 </div>

 <div class="card tw">
  <h3>Lotes cadastrados</h3>
  <table>
    <tr><th>Modelo</th><th>Banca</th><th>Qtd</th><th>Status</th><th></th></tr>
    <tr v-for="l in lotes" :key="l.id">
      <td>{{ l.modelo }}</td><td>{{ l.banca }}</td><td>{{ l.quantidade }}</td>
      <td>{{ l.status }}</td>
      <td><button @click="mostrarQr(l)">Ver QR</button></td>
    </tr>
  </table>
 </div>
</template>
```

Arquivo 36: `frontend/src/views/Leitor.vue`

```
<script setup>
import { ref, onUnmounted } from 'vue'
import { Html5Qrcode } from 'html5-qrcode'
import api, { msgErro } from '../api'

const msg = ref('')
const lote = ref(null)
const ativo = ref(false)
const manual = ref('')
const perda = ref({ tipo: 'DANIFICADA', quantidade: 1, observacao: '' })

let leitor = null
let travado = false

async function iniciar() {
 leitor = new Html5Qrcode('reader')
 try {
```

```
    await leitor.start({ facingMode: 'environment' }, { fps: 10, qrbox: 240 }, onLeitura)
    ativo.value = true
  } catch (e) {
    msg.value = 'Não foi possível abrir a câmera: ' + e
  }
}

async function parar() {
  if (leitor && ativo.value) { await leitor.stop(); ativo.value = false }
}

async function onLeitura(codigo) {
  if (travado) return
  travado = true
  await processar(codigo)
  setTimeout(() => { travado = false }, 3000)
}

async function processar(codigo) {
  try {
    const { data } = await api.get(`/lotes/${codigo.trim()}`)
    if (data.status === 'EM_PROCESSAMENTO') {
      lote.value = data
      msg.value = 'Registre as perdas (se houver) e confirme o retorno.'
      return
    }
    await confirmar(codigo.trim())
  } catch (e) {
    msg.value = msgErro(e)
  }
}

async function confirmar(codigo) {
  try {
    const { data } = await api.post(`/lotes/${codigo}/scan`)
    lote.value = data
    msg.value = `OK! ${data.modelo} agora está: ${data.status}`
  } catch (e) {
    msg.value = msgErro(e)
  }
}

async function registrarPerda() {
  try {
    await api.post(`/lotes/${lote.value.codigo}/perdas`, perda.value)
    lote.value = (await api.get(`/lotes/${lote.value.codigo}`)).data
    msg.value = 'Perda registrada.'
  } catch (e) {
```

```
    msg.value = msgErro(e)
  }
}

onUnmounted(parar)
</script>

<template>
  <div class="row">
    <div class="card">
      <h2>Leitor de QR Code</h2>
      <div id="reader"></div>
      <button v-if="!ativo" @click="iniciar">Ligar câmera</button>
      <button v-else @click="parar">Desligar câmera</button>
      <hr>
      <label>Sem câmera? Cole o código do lote
        <input v-model="manual" placeholder="código do lote">
      </label>
      <button @click="processar(manual)">Ler código</button>
    </div>

    <div class="card" v-if="lote">
      <h2>{{ lote.modelo }}</h2>
      <p>Banca: {{ lote.banca }} | Qtd: {{ lote.quantidade }}</p>
      <p>Status: <b>{{ lote.status }}</b> | Perdas: {{ lote.totalPerdas }}</p>

      <div v-if="lote.status === 'EM_PROCESSAMENTO'">
        <h3>Registrar perdas</h3>
        <select v-model="perda.tipo">
          <option value="DANIFICADA">Peça danificada</option>
          <option value="EXTRAVIADA">Peça extraviada</option>
          <option value="DEFEITO_FABRICACAO">Defeito de fabricação</option>
        </select>
        <input v-model.number="perda.quantidade" type="number" min="1">
        <input v-model="perda.observacao" placeholder="observação (opcional)">
        <button @click="registrarPerda">Registrar perda</button>
        <hr>
        <button @click="confirmar(lote.codigo)">Concluir retorno do lote</button>
      </div>
    </div>
  </div>
  <p class="card" v-if="msg">{{ msg }}</p>
</template>
```

PARTE D: Conferir e rodar

D.1 Conferir se criou tudo

No Explorer, abra as pastas e compare com o desenho da seção A.6. Contagem certa:

Pasta	Arquivos
`backend/` (pom.xml)	1
`backend/.../resources/`	1
`backend/.../rastreabilidade/` (Application)	1
`model/`	8
`repository/`	5
`dto/`	1
`service/`	2
`config/`	3
`web/`	2
Total backend	24
`frontend/` (package.json, vite.config.js, index.html)	3
`frontend/src/` (main.js, api.js, style.css, App.vue)	4
`frontend/src/router/`	1
`frontend/src/views/`	4
Total frontend	12

Total geral: 36 arquivos.

D.2 Rodar (resumo)

1. **XAMPP: Start no MySQL e rode o script SQL no phpMyAdmin (veja a Parte 2 da resposta anterior).**

Terminal 1 (Terminal > New Terminal):
cd backendmvn spring-boot:run

2. **Espere aparecer `Started RastreabilidadeApplication`.**

Terminal 2 (botão + no painel do terminal):
cd frontendnpm installnpm run dev

3.
4. **Abra http://localhost:5173 e entre com `gestor@senai.com` / `123456`.**

Se preferir, posso gerar a pasta inteira já pronta num arquivo .zip, para você só extrair e abrir no VS Code, sem criar os 36 arquivos à mão. Quer?